

Bloco 3: Ciclo de Vida CI/CD e Ferramentas

MÓDULO 3

O coração técnico do DevOps. Neste bloco, vamos entender o **Ciclo de Vida CI/CD** — a espinha dorsal da entrega contínua — e conhecer o ecossistema de ferramentas que torna tudo isso possível na prática.



Ciclo de Vida CI/CD

CI/CD é o conjunto de práticas que automatiza o caminho do código desde o desenvolvedor até o usuário final. Existem três níveis de maturidade nessa jornada:



Integração Contínua (CI)

Foco em **Build e Test**. O código é integrado ao repositório principal várias vezes ao dia. A cada integração, um pipeline automatizado compila o código e executa todos os testes. Falhas são detectadas em minutos, não em semanas.



Entrega Contínua (CD)

O código está **sempre pronto para ser deployado** em produção. O pipeline automatiza até o ambiente de staging/homologação, mas a decisão de ir para produção ainda pode ser manual — um "botão verde" que o time aperta quando quiser.



Implantação Contínua (CD)

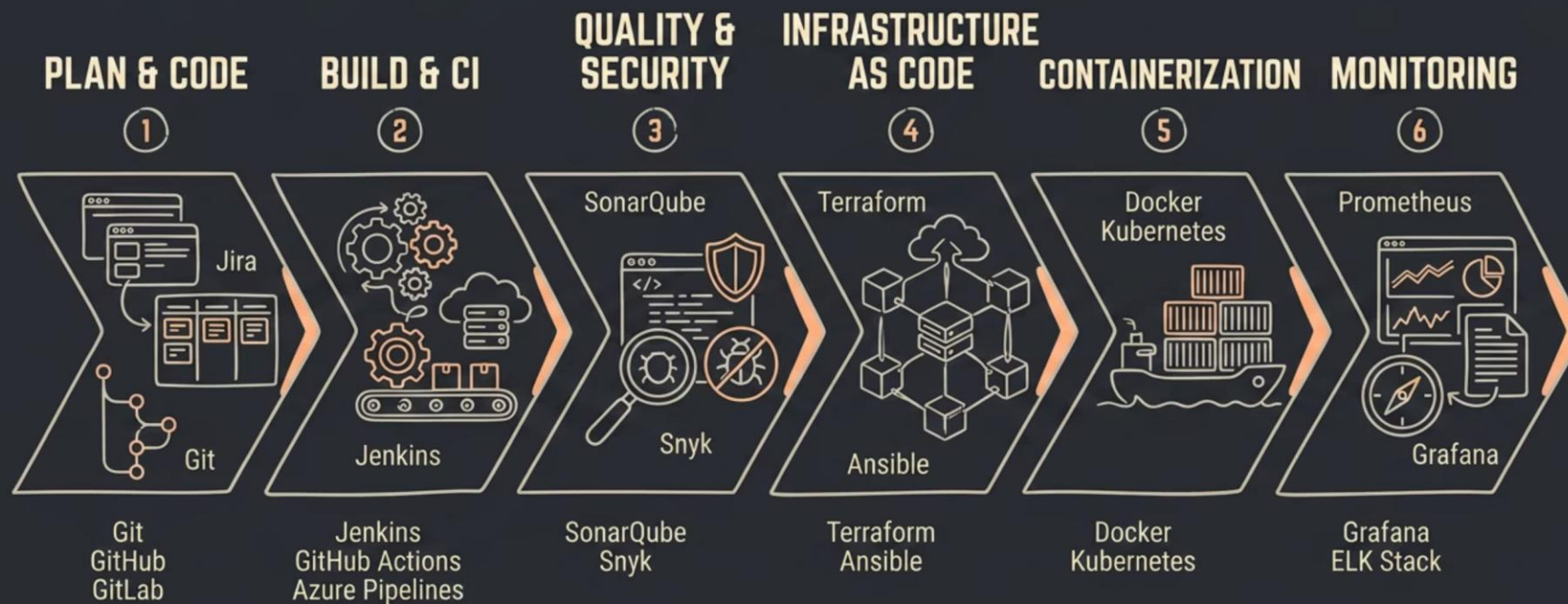
O processo é **100% automatizado até a produção**. Se o código passa em todos os testes, ele vai automaticamente ao ar — sem intervenção humana. Requer altíssima confiança nos testes e monitoramento robusto.



A maioria das empresas opera no nível de **Entrega Contínua**. A Implantação Contínua total é praticada por empresas como Netflix, Amazon e Google, que fazem milhares de deploys por dia.

O Ecossistema de Ferramentas DevOps

O mercado DevOps possui centenas de ferramentas. A chave é entender **qual ferramenta resolve qual problema** em cada estágio do pipeline. Veja o mapa completo:



Não é necessário dominar todas as ferramentas — o importante é entender o **propósito de cada categoria** e escolher as ferramentas certas para o contexto da sua organização.

Ferramentas: Planejamento, Código e CI

Planejamento & Código

- **Jira** — Gestão de tarefas, sprints e backlog. Integra com Git para rastrear código por issue.
- **GitHub** — Repositório de código, code review, pull requests e integração com Actions para CI/CD.
- **GitLab** — Plataforma completa: repositório + CI/CD integrado + gestão de projetos em uma só ferramenta.

Build & Integração Contínua

- **Jenkins** — O clássico open-source. Altamente configurável com milhares de plugins. Requer mais manutenção.
- **GitHub Actions** — CI/CD nativo do GitHub. Configurado via YAML, fácil de começar, gratuito para projetos open-source.
- **GitLab CI** — CI/CD integrado ao GitLab. Pipelines definidos no arquivo `.gitlab-ci.yml`.
- **Azure Pipelines** — Solução da Microsoft, ideal para ecossistemas Azure e .NET.

Ferramentas: Qualidade, Segurança e IaC



Qualidade de Código — SonarQube

Analisa o código-fonte em busca de bugs, vulnerabilidades, code smells e dívida técnica. Integra ao pipeline de CI e bloqueia merges que não atendem aos critérios de qualidade definidos pelo time. Suporta mais de 30 linguagens de programação.



Segurança — Snyk

Especializado em segurança de dependências (bibliotecas de terceiros). Identifica vulnerabilidades conhecidas (CVEs) nas dependências do projeto e sugere versões corrigidas. Integra ao GitHub, GitLab e pipelines CI/CD. Conceito de **DevSecOps** — segurança desde o início.



Infraestrutura como Código (IaC)

Terraform — Provisiona infraestrutura em qualquer cloud (AWS, Azure, GCP) usando código declarativo HCL. "Descreva o que você quer, não como fazer."

Ansible — Automatiza configuração de servidores e deploys de aplicações usando playbooks YAML. Sem agente instalado nos servidores.

Ferramentas: Containerização

A containerização revolucionou a forma como aplicações são empacotadas e executadas, eliminando o clássico problema *"funciona na minha máquina"*.

Docker

Empacota a aplicação e todas as suas dependências em um **container** — uma unidade isolada e portátil que roda de forma idêntica em qualquer ambiente.

- Dockerfile define o ambiente
- Docker Hub como registry de imagens
- Docker Compose para múltiplos containers localmente
- Base para qualquer estratégia de containerização

Kubernetes (K8s)

Orquestra centenas ou milhares de containers em produção, gerenciando automaticamente disponibilidade, escalabilidade e recuperação de falhas.

- Auto-scaling baseado em carga
- Self-healing: reinicia containers com falha
- Rolling updates sem downtime
- Gerenciamento de secrets e configurações
- Disponível como serviço gerenciado: EKS, AKS, GKE



Ferramentas: Monitoramento e Observabilidade

ESTÁGIO FINAL DO PIPELINE

Monitoramento não é opcional — é o que fecha o loop de feedback do DevOps. Sem visibilidade do que acontece em produção, não há como melhorar continuamente.



Prometheus

Sistema open-source de coleta e armazenamento de métricas. Usa um modelo de **pull** — ele busca métricas dos serviços em intervalos regulares. Altamente escalável e com linguagem de query poderosa (PromQL).



Grafana

Plataforma de visualização de dados. Conecta ao Prometheus (e outras fontes) para criar **dashboards ricos e interativos**. Permite configurar alertas visuais e compartilhar painéis com toda a organização.



ELK Stack

Elasticsearch + Logstash + Kibana. Coleta, processa, armazena e visualiza **logs** de todos os serviços. Essencial para investigar incidentes, rastrear erros e entender o comportamento da aplicação em produção.

✓ A tríade de observabilidade moderna é: **Métricas** (Prometheus/Grafana) + **Logs** (ELK Stack) + **Traces** (Jaeger/Zipkin). Juntos, eles dão visibilidade completa do sistema.